camp tocamp
INNOVATIVE SOLUTIONS
BY OPEN SOURCE EXPERTS

Odoo avec des agents

Comment je travaille sur Odoo avec Codex et Claude, du premier email du client à la livraison

Camptocamp · novembre 2026

Sur Odoo, l’IA est désormais plus forte que moi

« Il n’y a plus aucun domaine Odoo où l’IA n’est pas plus forte que moi. Quand je me dis que ce n’est pas vrai, c’est qu’il lui manque quelque chose. Le reste, c’est de l’ego. »

Quand l’agent se trompe, je cherche d’abord ce qui lui manque

1. **La configuration**
   Il n’a pas les bons outils, les bons droits ou le bon modèle.

2. **Le contexte**
   Il n’a pas la demande complète, l’historique du projet ou les décisions prises.

3. **Les outils**
   Il ne peut pas tester : pas de copie locale, pas de navigateur.

camptocamp
INNOVATIVE SOLUTIONS
BY OPEN SOURCE EXPERTS

Essayer va souvent plus vite que débattre

Ce que j’observe

Discuter longtemps d’un développement ou d’un Studio, même pour valider une hypothèse avec le client, prend souvent plus de temps que de le faire.

Alors on le fait, on le montre, et on décide. Surtout en local, sur un Docker Compose.

Une copie locale de la staging

Neutralisée : aucun email ni aucune donnée ne part chez le client.

La production en lecture seule

Chaque modification en production est validée une par une.

Pas de test refait pour rien

On ne relance un test que si quelque chose a changé.

Une personne responsable

Elle suit les critères, les risques et les preuves jusqu’au bout.

Pourquoi on peut essayer sans risque

camptocamp
INNOVATIVE SOLUTIONS
BY OPEN SOURCE EXPERTS

Le métier de consultant change

L'agent fait le travail. Il me reste ce qu'il ne peut pas faire seul : ne rien perdre de ce que dit le client, fixer le but, et vérifier comme un utilisateur.

1

Capter
Ne rien perdre de ce que dit le client

→

2

Cadrer
Fixer les critères, les dépendances et le temps

→

3

Réaliser
L'agent travaille, le pack se rejoue

→

4

Vérifier
Dans le navigateur, avec des personas

→

5

Consigner
Tout garder dans Git

Pour illustrer : Boréal, un client fictif, nous écrit au sujet de ses factures d'acompte. On suit sa demande du premier email jusqu'à la mémoire du projet.

camptocamp
INNOVATIVE SOLUTIONS BY OPEN SOURCE EXPERTS

Config, Studio ou développement, c’est le même travail

Config
Paramétrage standard

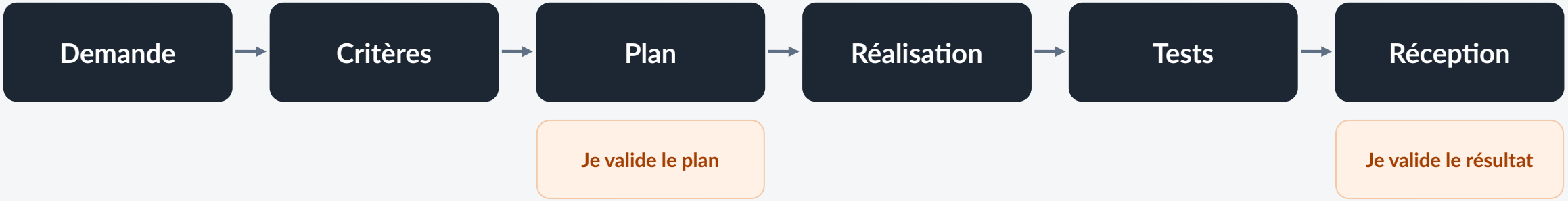
On vérifie que le processus métier fonctionne.

Studio
Champs, vues, automatisations

On rejoue le pack Studio et on dit ses limites.

Développement
Module custom

On vérifie les tests, les droits et l’installation.



Seuls les tests et les critères d’acceptation changent. Les étapes restent les mêmes, et c’est toujours un humain qui valide le plan, le résultat et chaque modification en production.

camp to camp
INNOVATIVE SOLUTIONS BY OPEN SOURCE EXPERTS

Ne rien perdre de ce que dit le client

Slack
Une décision prise dans un fil et jamais reportée.

Mon PC
Un fichier, un export ou une note restés en local.

Le bilatéral
Une discussion avec un collègue que personne n'a notée.

La réunion
Ce qui a été dit, mais jamais écrit.

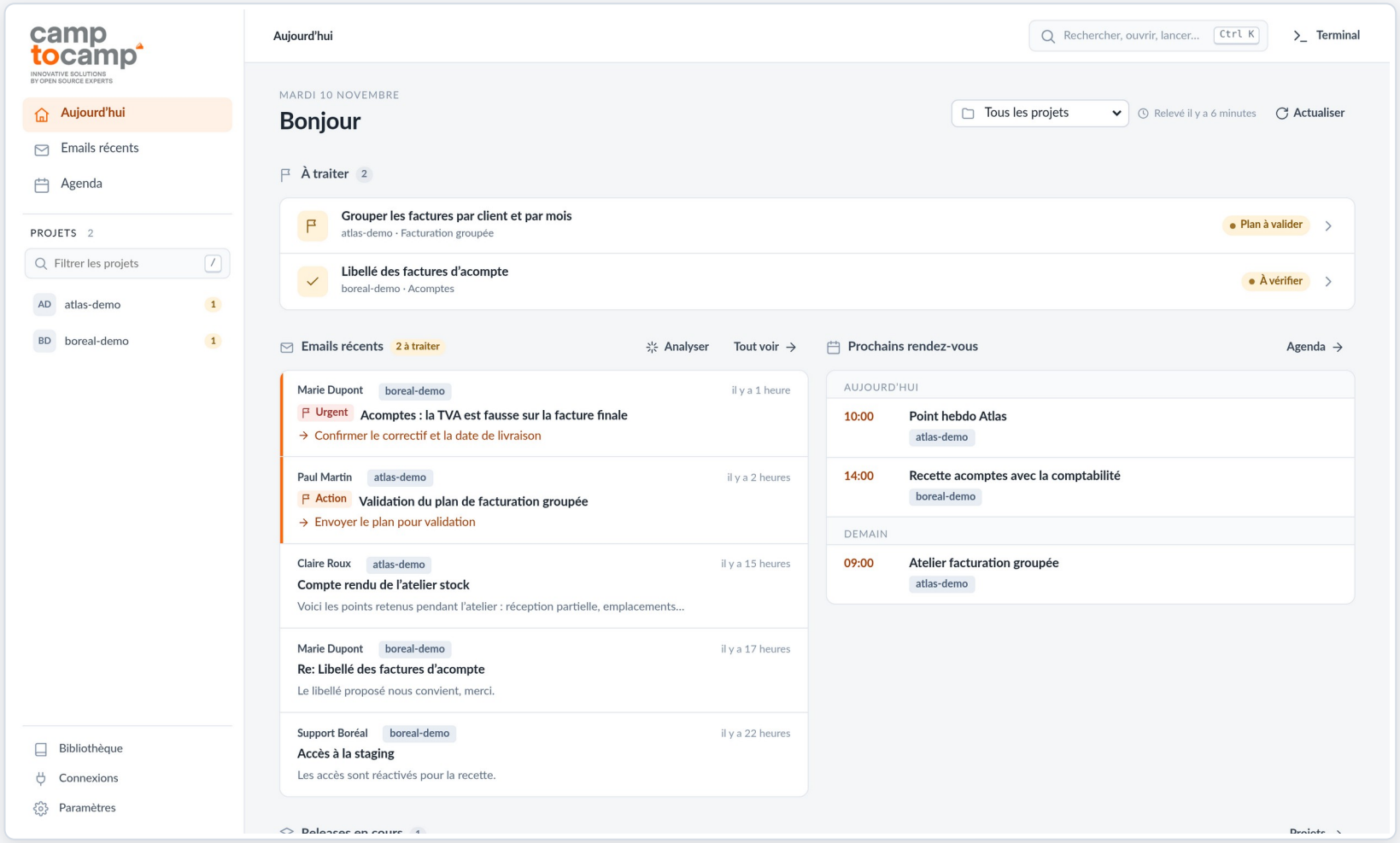
Tout revient dans le dépôt du client

C'est peut-être le plus important du métier. Après chaque réunion, je me demande : est-ce que j'ai perdu quelque chose ?

- L'email du client, tel quel, dans la release
- Les décisions et leurs raisons, dans la release
- Une ligne de journal à chaque intervention
- Le compte rendu dans le dépôt, pas sur mon PC

camp to camp
INNOVATIVE SOLUTIONS BY OPEN SOURCE EXPERTS

Le matin, je vois d’abord ce qui attend une décision



1. **À traiter**
   Les plans à valider et les résultats à vérifier, tous projets confondus.

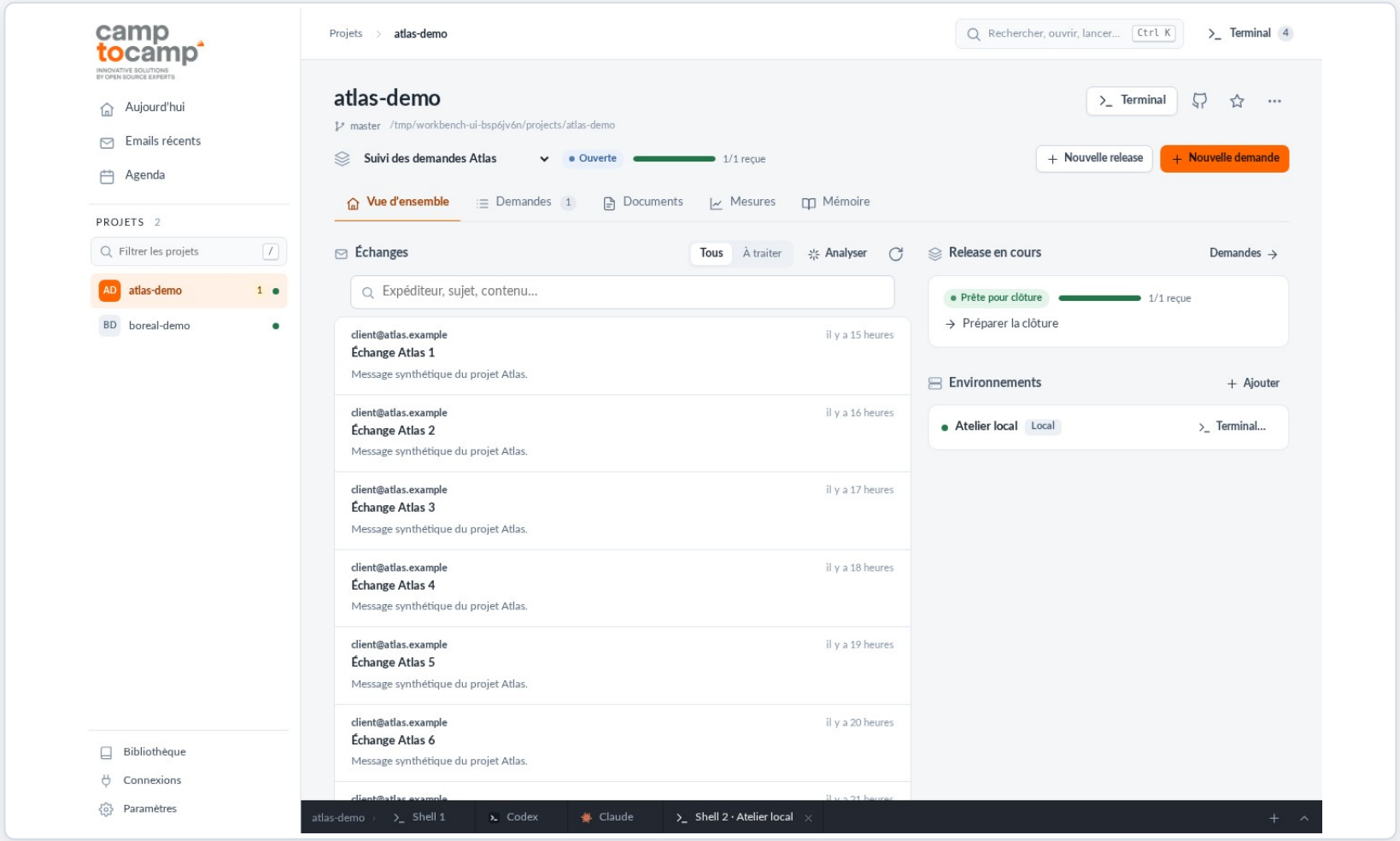
2. **Les emails importants**
   Codex ou Claude repère ceux qui demandent une action.

3. **Le nom du projet**
   Sur chaque email et chaque rendez-vous.

4. **Un filtre par projet**
   Le même sur l’accueil, les emails et l’agenda.

camp to camp
INNOVATIVE SOLUTIONS BY OPEN SOURCE EXPERTS

Tous les échanges d'un client au même endroit



1 **50 échanges par projet**
Les 50 derniers échanges, séparés des 50 emails généraux.

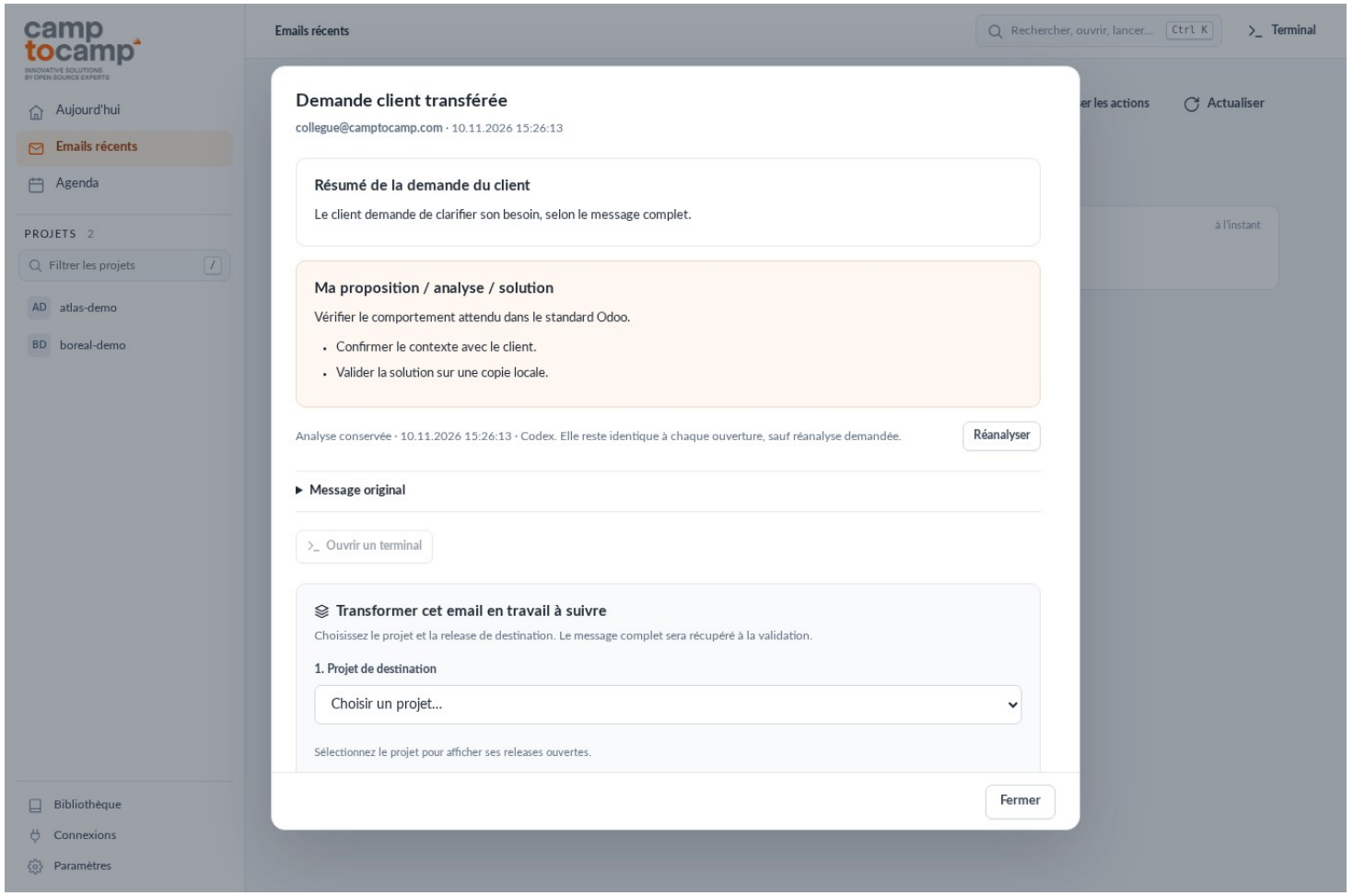
2 **Une recherche**
Par expéditeur, sujet ou contenu.

3 **Les emails à traiter**
Ceux qui attendent une réponse.

4 **Un terminal sur le projet**
Codex ou Claude, dans le bon dépôt.

camp tocamp
INNOVATIVE SOLUTIONS BY OPEN SOURCE EXPERTS

Lire un email : résumé et proposition



1. **Résumé client**
   La demande, le contexte et les échéances explicites.

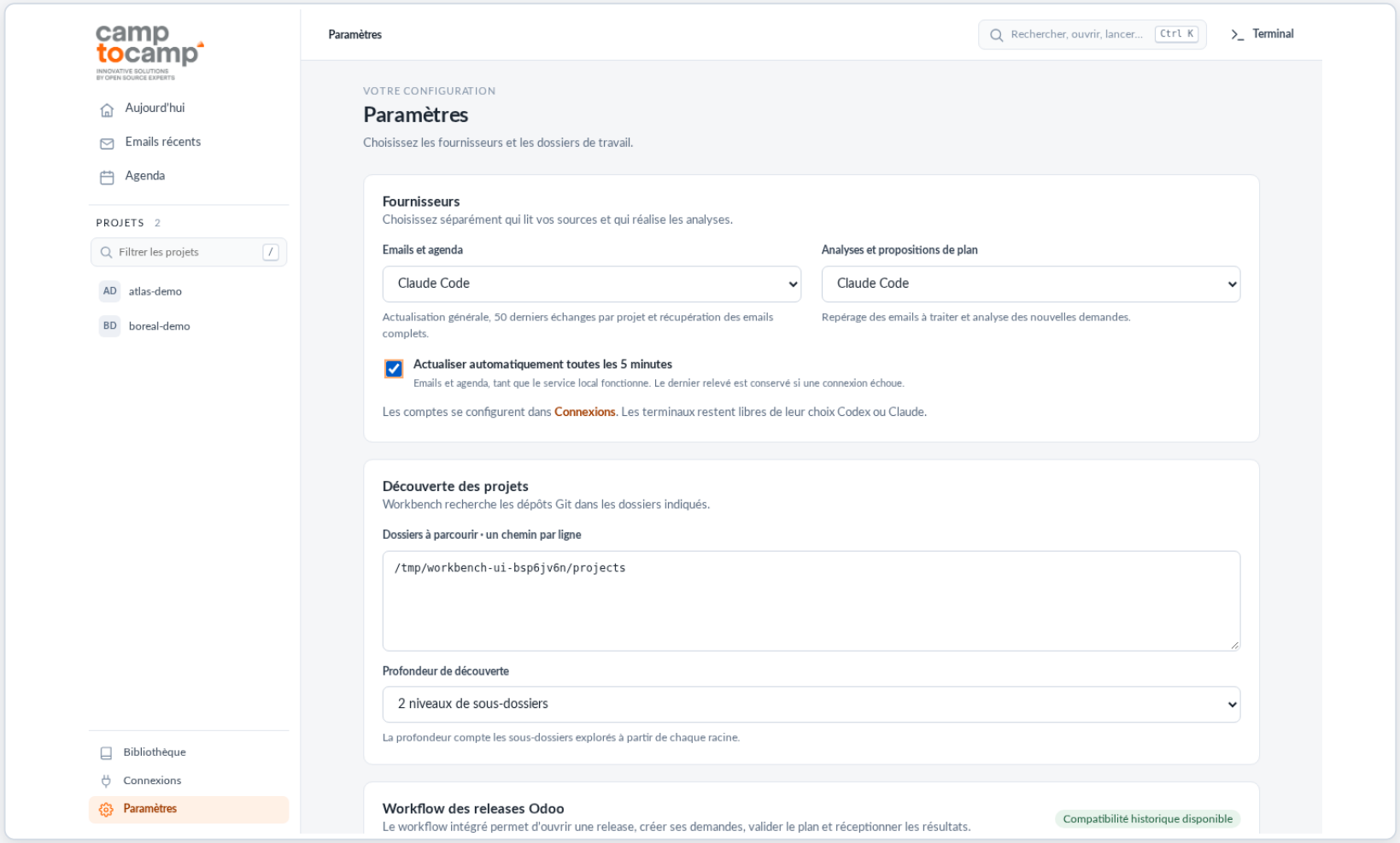
2. **Ma proposition**
   Piste de solution, hypothèses et prochaine action.

3. **Analyse enregistrée**
   Fermer, actualiser ou redémarrer conserve le résultat.

4. **Réanalyser**
   Une nouvelle analyse seulement après ce clic explicite.

camp tocamp
INNOVATIVE SOLUTIONS BY OPEN SOURCE EXPERTS

Emails et agenda actualisés automatiquement



1. **50 emails + 50**
   50 derniers reçus ; 50 échanges propres à chaque projet.

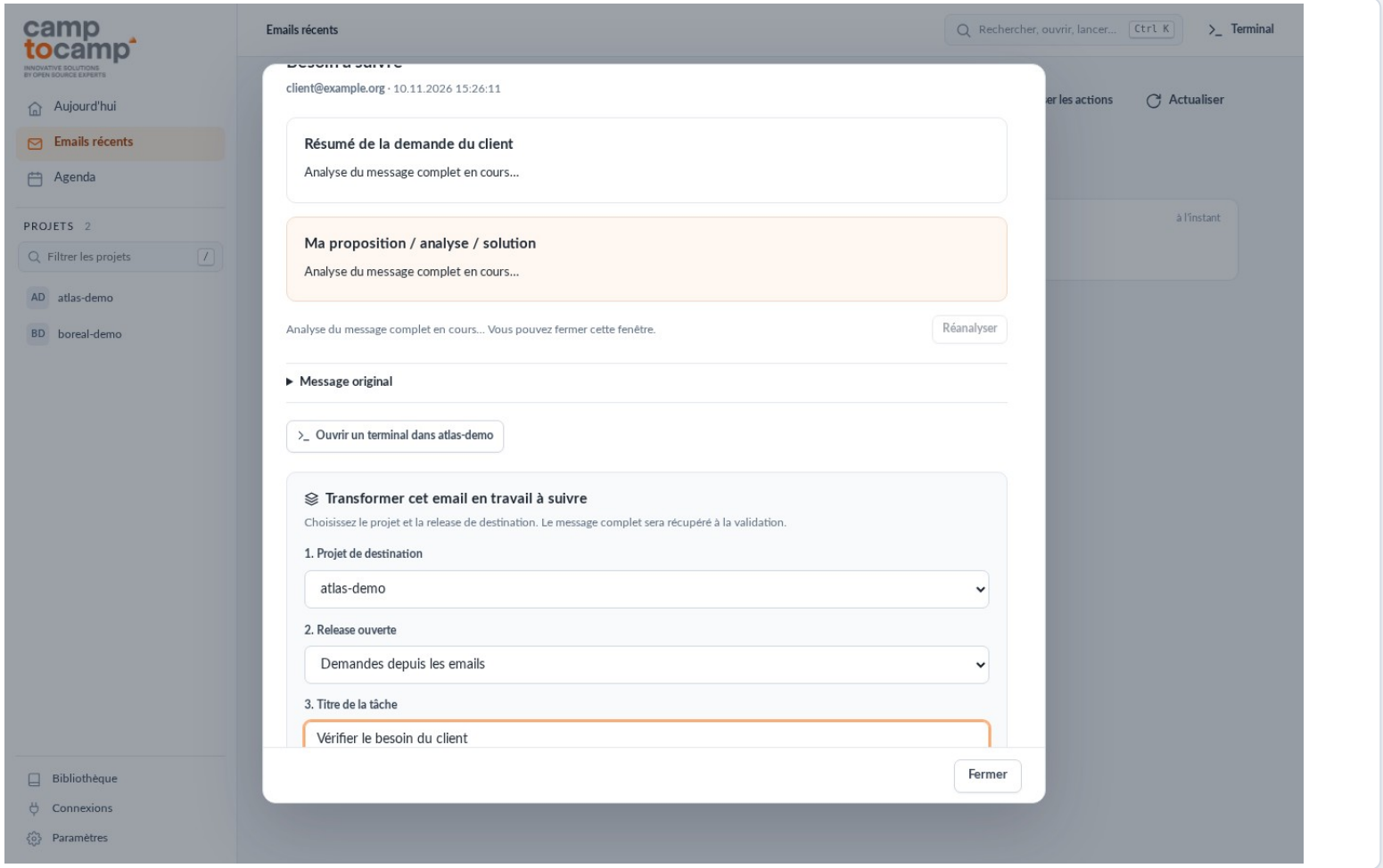
2. **Toutes les 5 minutes**
   Emails et agenda, tant que le service fonctionne.

3. **Le fournisseur choisi**
   Codex ou Claude, séparément pour sources et analyses.

4. **Lecture seule**
   La relève ne crée aucune tâche et n'analyse aucun email.

camp to camp
INNOVATIVE SOLUTIONS BY OPEN SOURCE EXPERTS

Un clic, et l'email rejoint la release



1 **L'email complet**
Ajouté tel quel à la demande, sans être réécrit.

2 **La release ouverte**
Choisie explicitement si plusieurs releases sont ouvertes.

3 **Pas de doublon**
Un email déjà ajouté ne l'est pas une seconde fois.

4 **Lu par les agents**
Ils partent de la demande d'origine du client.

camp to camp
INNOVATIVE SOLUTIONS BY OPEN SOURCE EXPERTS

Je passe mon temps sur les critères et les dépendances

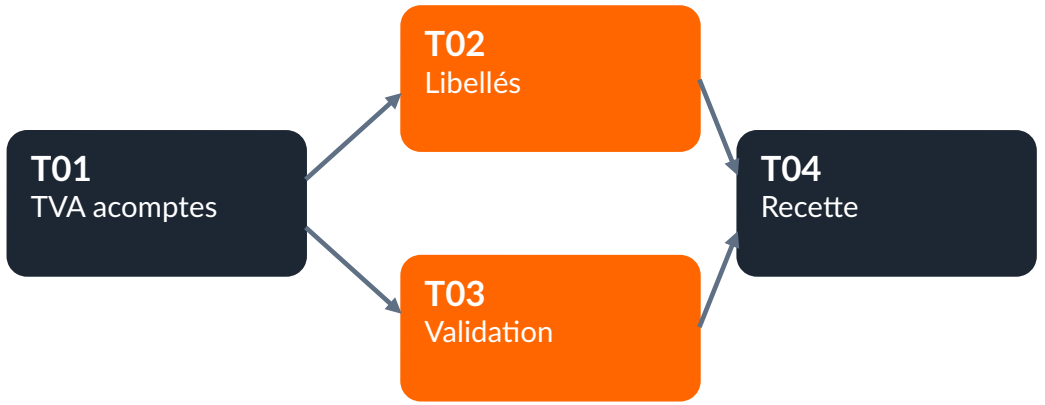
Critères d'acceptation

C'est d'eux que dépend la qualité, qu'il s'agisse d'une migration, d'un développement ou d'une config. Ils disent ce qu'on veut obtenir et à quoi on verra que c'est réussi.

- Valider le plan, c'est d'abord valider les critères.
- Ne pas forcer l'agent dans une direction, ou alors lui expliquer pourquoi.
- Un critère se prouve dans le métier, pas par un test qui passe.

Dépendances entre tâches

Le travail d'un agent peut prendre du temps. Ce qui ne dépend de rien part en parallèle, avec des sous-agents.



T02 et T03 en parallèle

camp to camp
INNOVATIVE SOLUTIONS BY OPEN SOURCE EXPERTS

On estime le temps des agents, puis on le mesure

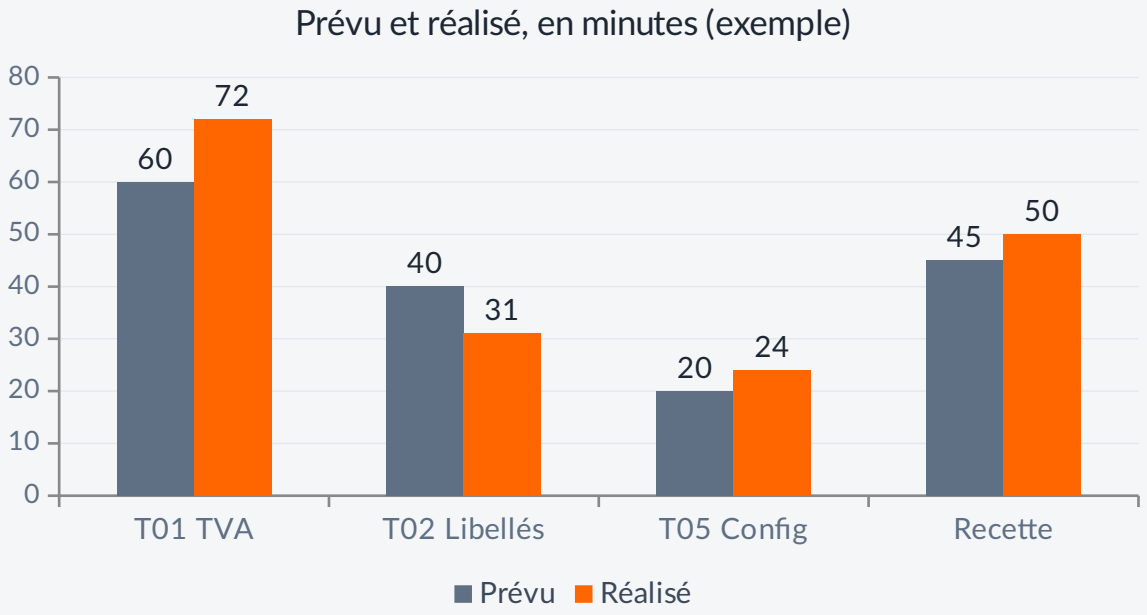
Trois chiffres par tâche et par agent

Optimiste, probable et pessimiste, puis une moyenne pondérée :

$$(O + 4 \times P + Pe) / 6$$

Exemple : 30, 50 et 90 minutes donnent 53 minutes

On compare avec des releases déjà mesurées. Sans comparaison possible, l'estimation est signalée comme peu fiable.



Pas d'estimation, pas de démarrage

Ensuite, le temps réel est mesuré tâche par tâche.

On garde la première estimation

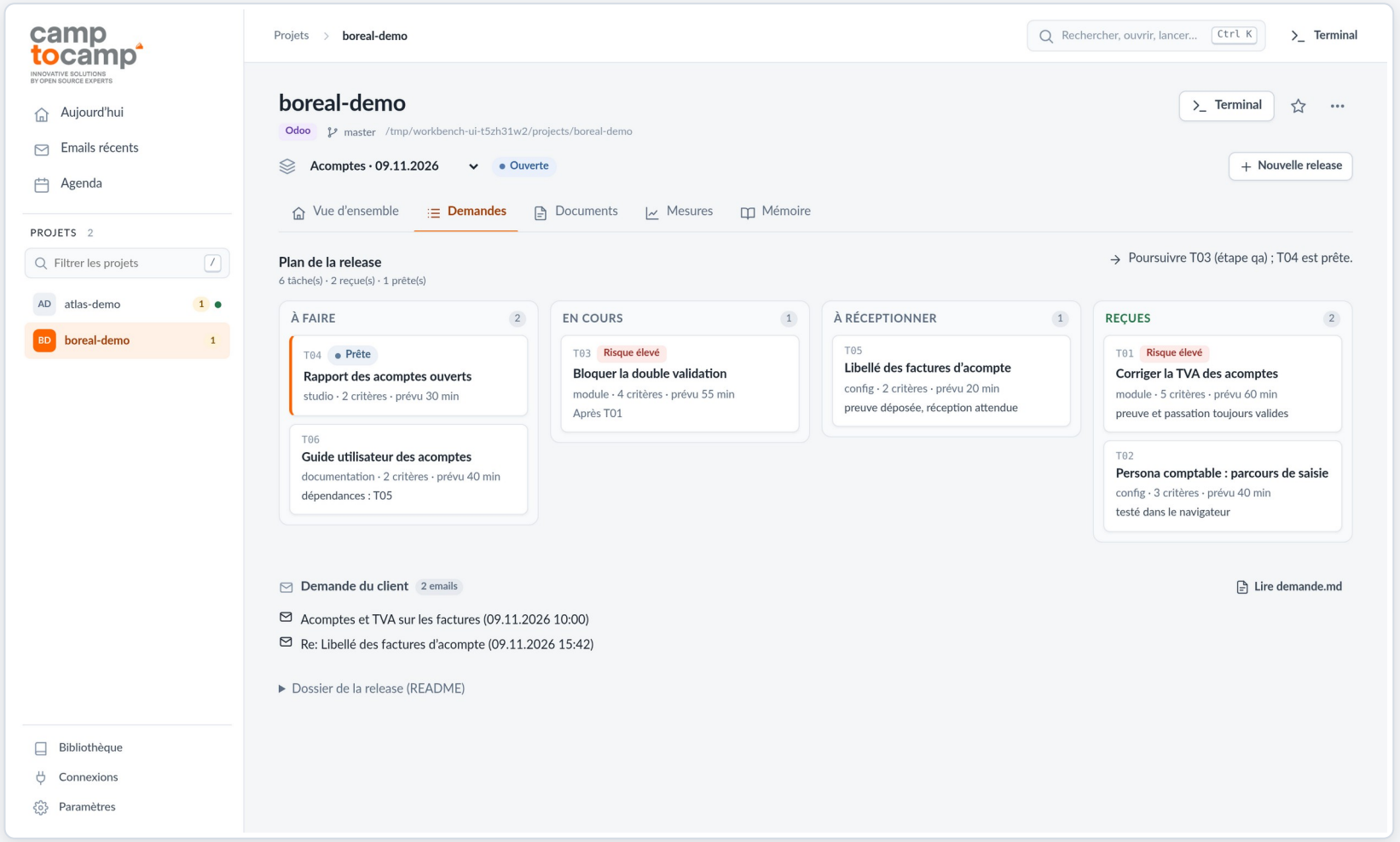
Si le périmètre change, on la révise en disant pourquoi.

Le temps des agents uniquement

Ni temps humain ni prix. Et le total n'est pas un délai : les tâches se chevauchent.

camptocamp
INNOVATIVE SOLUTIONS
BY OPEN SOURCE EXPERTS

Le plan de la release se lit comme un kanban



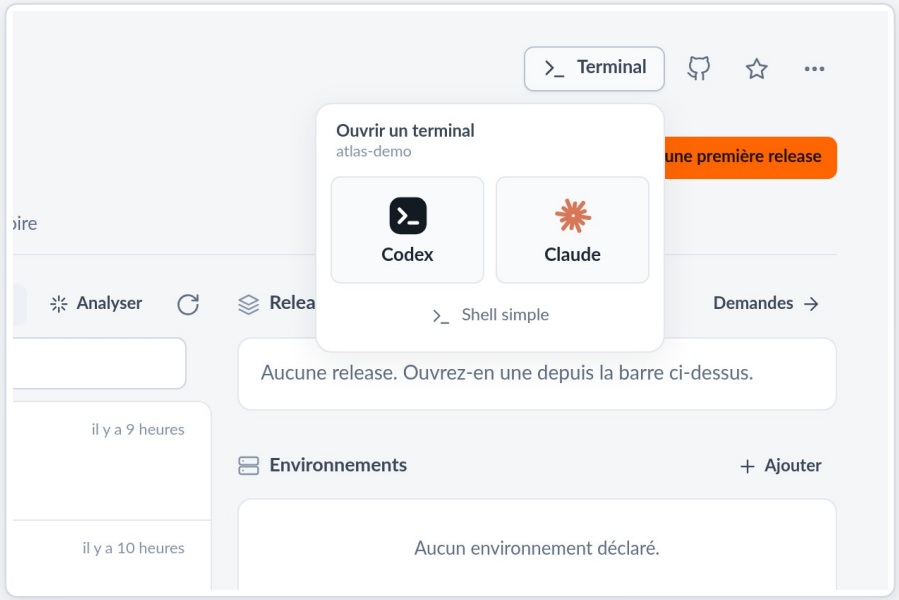
1 **Les tâches du plan**
Lues directement dans le dépôt, sans rien modifier.

2 **Où en est chaque tâche**
À faire, en cours, à valider ou terminée.

3 **La prochaine étape**
Écrite en une phrase, en haut du tableau.

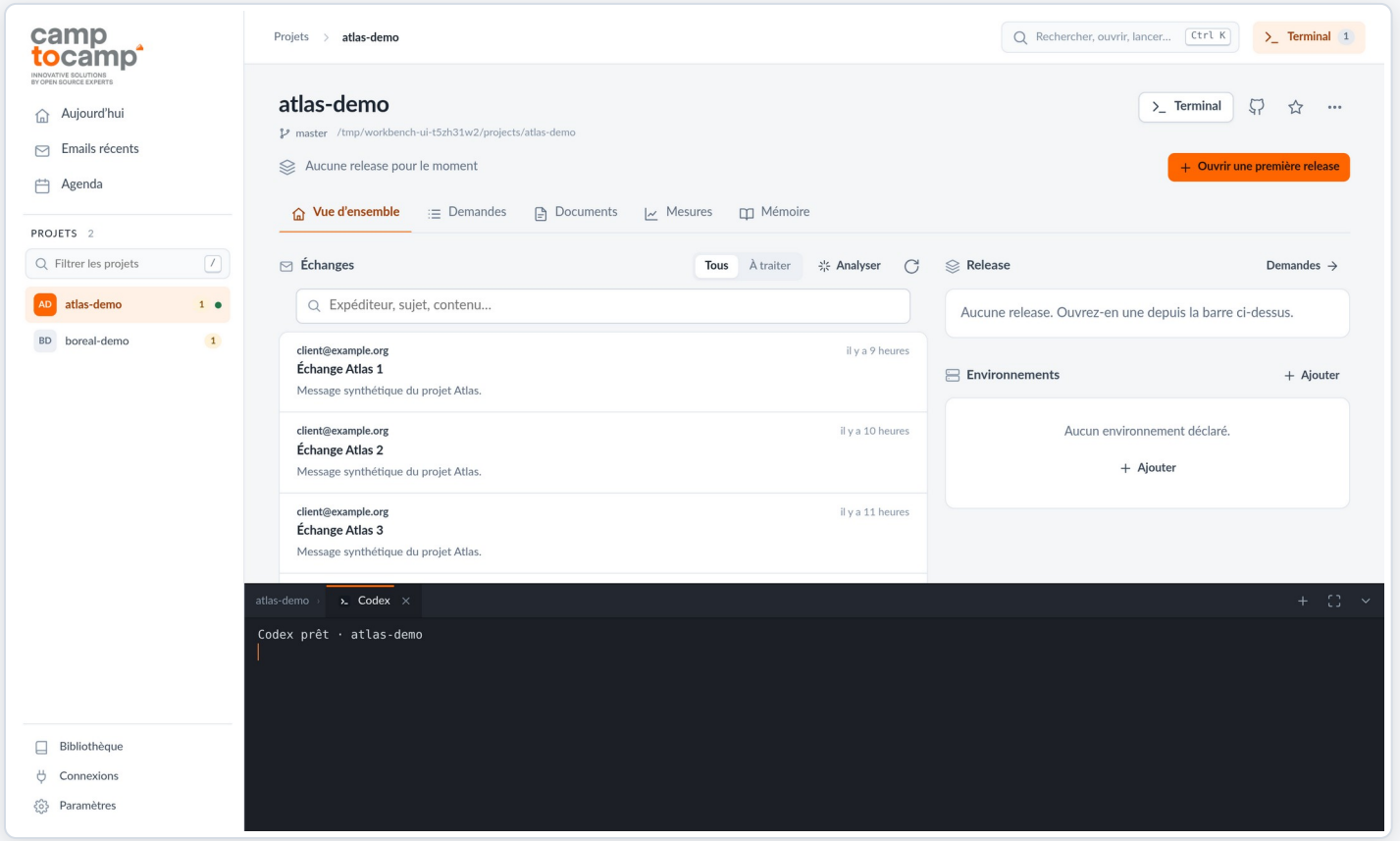
4 **La demande du client**
Les emails ajoutés, juste en dessous.

L’agent travaille dans le dépôt du projet



Codex ou Claude, au choix

Le terminal s’ouvre directement dans le dépôt du projet. Pas de chat, seulement des terminaux.



Toujours sous la main

Même réduit, un onglet signale quand l’agent a écrit quelque chose.

camp to camp
INNOVATIVE SOLUTIONS BY OPEN SOURCE EXPERTS

Le pack testé est exactement celui qu'on livre

Pack de config | Pack Studio | Pack de dev standard

Rangé dans Git, il se rejoue à l'identique

1

On copie la staging
Une copie très récente, en local, sur Docker Compose

→

2

On applique le pack et on teste
Critères d'acceptation, navigateur, personas

→

3

Le développement prend du temps
Pendant ce temps, la staging et la production évoluent

→

4

La dernière version du pack part sur la master
Ce qu'on a testé est ce qu'on livre

Config, Studio ou développement, on livre toujours un pack. On ne refait jamais à la main en production ce qu'on a validé sur la copie.

camptocamp
INNOVATIVE SOLUTIONS BY OPEN SOURCE EXPERTS

Tester comme un utilisateur, dans le navigateur

Pourquoi le navigateur

Dans Odoo, un écran est construit à partir de plusieurs héritages de vues. L'agent modifie un morceau sans voir l'écran final, et il juge mal l'ergonomie.

- Ouvrir le vrai écran, sur la copie locale
- Vérifier l'ergonomie écran par écran
- Tester les droits d'accès avec chaque persona

En début de projet, je crée deux ou trois personas qui n'attendent pas la même chose du même processus

C

Comptable
Saisir vite, sans erreur de TVA, et ne jamais valider deux fois.

R

Responsable
Voir les écarts et décider, sans rien saisir.

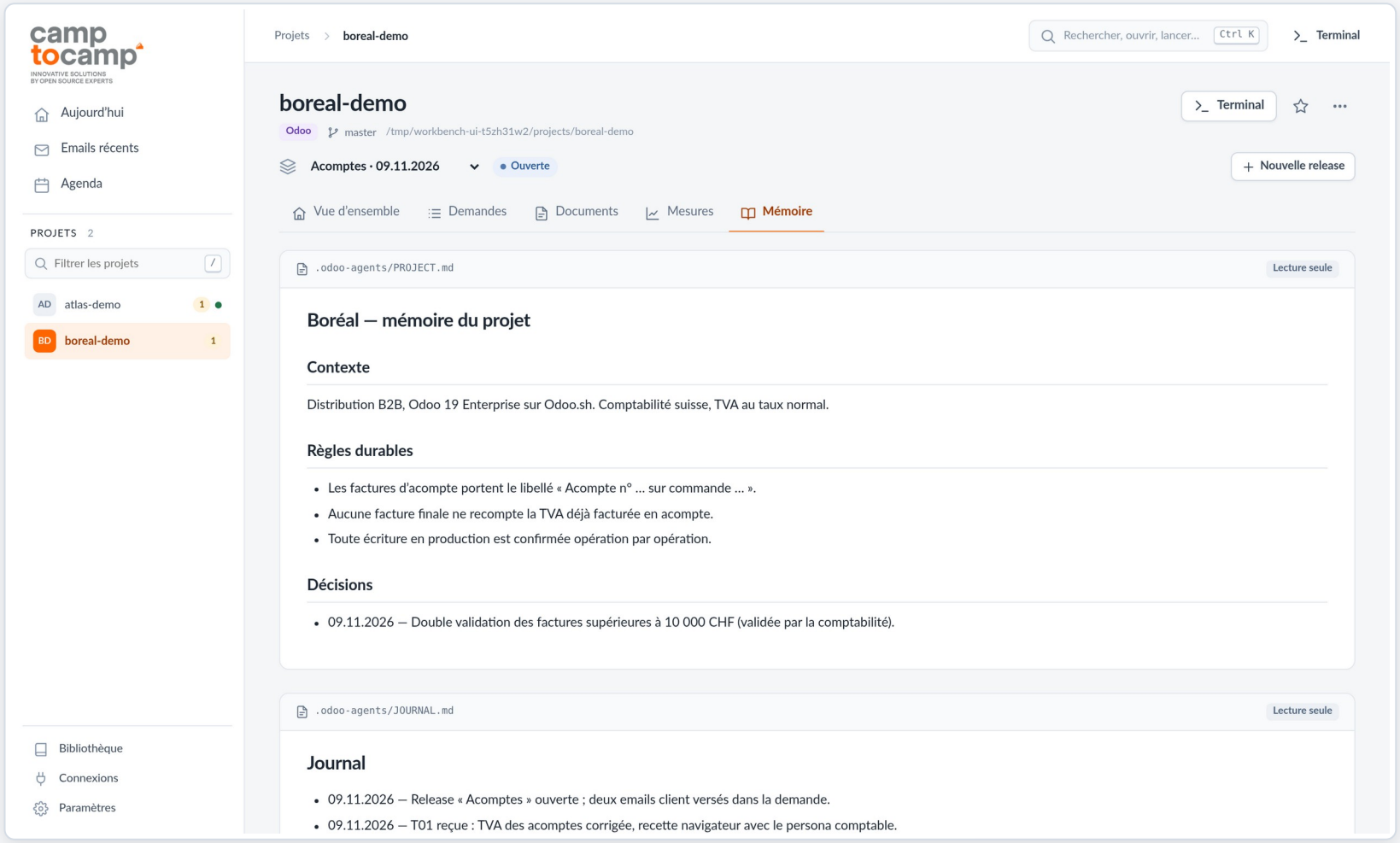
C

Commercial
Facturer l'acompte sans toucher à la comptabilité.

Exemple : les factures d'acompte chez Boréal

camptocamp
INNOVATIVE SOLUTIONS
BY OPEN SOURCE EXPERTS

La mémoire du projet est dans le dépôt du client



1 **Le projet**
Les règles, les décisions et un journal court.

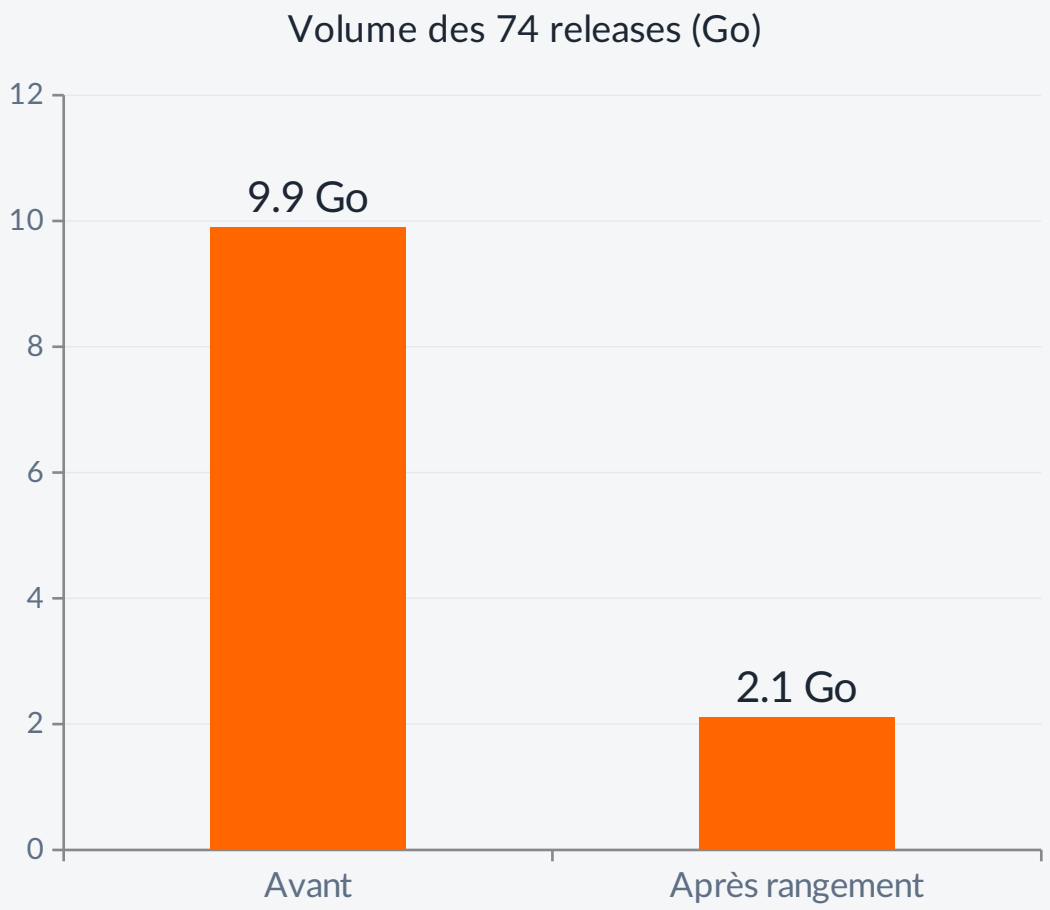
2 **La release**
La demande, le plan, la recette et les preuves.

3 **Un briefing ciblé**
L'agent lit toutes les décisions, et le reste selon le besoin.

4 **Les retours**
Collectés à chaque clôture, testés avant de devenir une règle.

camp to camp
INNOVATIVE SOLUTIONS BY OPEN SOURCE EXPERTS

Des releases plus légères



Ce qui reste dans la release

- La demande, le plan, la recette, la documentation
- Des preuves courtes, les captures et les pièces du client
- Le bilan de clôture

Ce qui part dans un dossier de travail

- Les essais, les copies, les journaux bruts, les archives
- Hors de Git, et rien n'est supprimé
- 97 fichiers par release en médiane, 69 après

camptocamp
INNOVATIVE SOLUTIONS
BY OPEN SOURCE EXPERTS

Mon agent à moi, un savoir commun à toute la société

Mon agent orchestrateur
Je suis le seul à l'utiliser. Il connaît mon historique, et je l'enrichis au fil des projets.

Le harnais, OpenClaw et Workbench
Codex et Claude y sont interchangeables : les mêmes rôles et les mêmes commandes pour les deux.

Commun à toute la société
La base documentaire, les skills, les rôles et la bibliothèque RH, paie et compta.
Les skills sensibles restent réservés à ceux qui en ont besoin.

Pourquoi un harnais
- Construire l'intelligence de notre entreprise, pas celle d'un éditeur
- Ne dépendre ni de Codex, ni de Claude, ni de leurs outils
- Exporter la mémoire professionnelle de Claude et de Codex vers le harnais marche très bien

camptocamp
INNOVATIVE SOLUTIONS
BY OPEN SOURCE EXPERTS

Un collègue démarre sans agents privés

Inclus dans le dépôt

Dashboard et workflow intégré : demandes, plan à valider et réception.

À connecter sur son poste

Codex ou Claude, puis ses connecteurs Gmail et Google Calendar.

Pour les projets Odoo

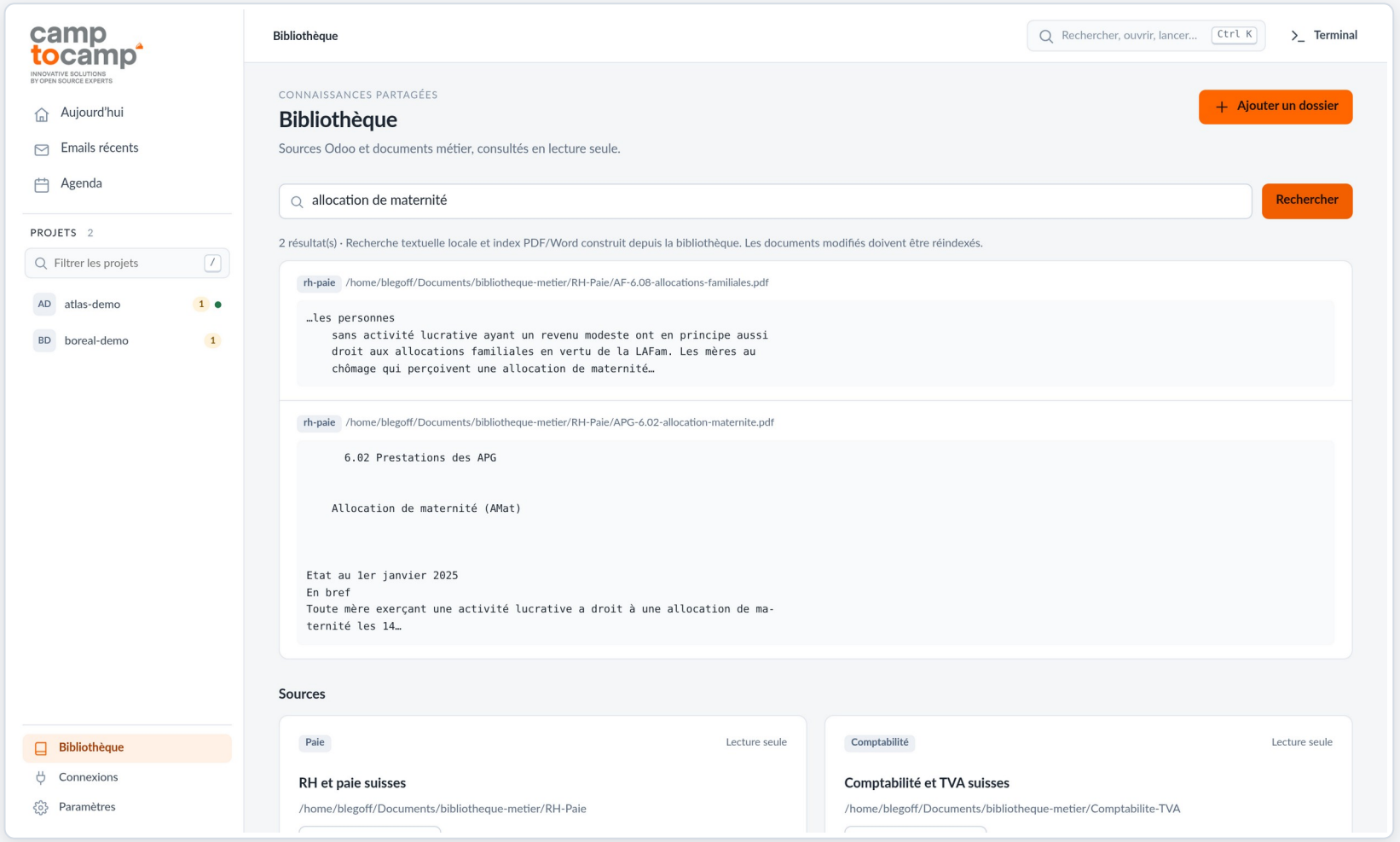
Les sources, bases et packs métier restent propres au projet. Les anciennes releases gardent leur moteur.

Installation autonome

- README et prérequis fournis
- Vérifier les dépendances avant le démarrage
- Guide pratique PDF inclus

camptocamp
INNOVATIVE SOLUTIONS
BY OPEN SOURCE EXPERTS

Une bibliothèque que les agents consultent



1 **Des sources officielles**
Mémentos AVS, LPP et LAA, documents de l'AFC, QR-facture.

2 **Une recherche dans les PDF**
Un index local, en lecture seule.

3 **Les agents y ont accès**
Claude et Codex la consultent en ligne de commande.

4 **Une source, pas un ordre**
On la cite avec sa date de validité.

camp to camp
INNOVATIVE SOLUTIONS BY OPEN SOURCE EXPERTS

Réduire l’écart entre profils techniques et non techniques

Sans formation, l’écart entre profils techniques et non techniques ne se comblera pas. Et comme le travail va très vite, il faut tout noter, donc passer par Git.

1

Docker
Savoir ce qu’est un conteneur et lancer la copie d’un client avec Docker Compose.

2

Git
Enregistrer, comparer, revenir en arrière, partager une release.

3

Le terminal
Ouvrir Codex ou Claude dans le bon dépôt et suivre ce qu’il fait.

C’est le minimum pour tout le monde, consultants compris. L’outil aide : projets, emails et terminaux au même endroit.

camptocamp
INNOVATIVE SOLUTIONS
BY OPEN SOURCE EXPERTS

Soyons ambitieux, et refactorisons sans hésiter

- Une décision d’architecture ne tient plus ? On refactorise.
- Avec les modèles actuels, refactoriser devient presque de la maintenance.
- L’IA peut aider partout dans un projet : il faut chercher où.

EXEMPLE FICTIF : BORÉAL

Refactoriser
Une architecture difficile à maintenir mérite une reprise avant d’ajouter une fonction.

EXEMPLE DE MÉTHODE

Reprise de données
Préparer, tester et rapprocher les données sur une copie locale avant de livrer.

camptocamp
INNOVATIVE SOLUTIONS
BY OPEN SOURCE EXPERTS

camp to camp
INNOVATIVE SOLUTIONS
BY OPEN SOURCE EXPERTS

À retenir

1. Quand l'agent échoue, il lui manque de la config, du contexte ou des outils. Le reste, c'est de l'ego.
2. Essayer en local plutôt que débattre.
3. Ne rien perdre : tout ce que dit le client revient dans Git.
4. Passer son temps sur les critères, les dépendances et l'estimation.
5. Livrer le pack qu'on a testé, et tester dans le navigateur avec des personas.
6. Un agent personnel, un savoir commun, et ne jamais hésiter à refactoriser.